

修改记录:

2015-10-20

1. 编写新版程序下传数据格式

1 下传数据格式

1.1 数据结构

下传的数据格式如 stMcCmndType 结构体所示，头 4 个字节为数据头；接下来的若干字节是数据；数据区根据不同的指令，有不同的型式。

```
struct stMcCmndType
{
    char u8SpCom_Headr; //该位固定为 0xAA;
    char u8SpCom_DtLen; //数据区长度;
    char u8SpCom_CardId; //保刘，写 0
    char u8SpCom_Check; //数据长度按位取反;
    // 上面四个字节为帧，下面为数据区

    char u8SpDt_CheckSum; //数据区校验和，不包括帧头;
    char u8SpDt_CmndType; //指令类型，表格 1
    char u8SpDt_Len; //数据区长度
    char u8SpDt_CmndNo; //下传指令编号，每下传指令自增 1 即可，无实际意义;
    char u8SpDt_Buf[0x80]; //数据区，最大长度为 80 个字节，对于不同的指令，该区内容不同
```

}

表格 1 下传指令与数据区格式

序号	u8SpDt_CmndType	u8SpDt_Buf 缓冲区的格式	说明
1	0xF2 设置轴参数	struct stSetSysParaCmdTyp{ uint8 AxisID; uint8 ParaIdx; float fPara; }*pSetParaCmd;	AxisD: 轴号 0、1、2、3 ParaIndex: 参数号 0-7 fPara: 参数值
2	0xF5 读参数	struct stSetSysParaCmdTyp{ uint8 AxisID; uint8 ParaIdx; }*pReadParaCmd;	AxisD: 轴号 0、1、2、3 ParaIndex: 参数号 0-7
3	0xF4 手动操作	struct stMcCmnd_ManulOpTyp{ uint8 SubCmnd; uint8 AxisID; int8 SpdDir; uint8 dummy; float PosCmd; float VelCmd; float AccCmd; }*pMcOpCmnd;	SubCmnd: 子命令号 0x00: 手动移动各个轴参数 规定距离 0x01: 增量方式移动, 用轴 参数定义的速度和加速度 0x03: 增量方式移动, 用下 传的参数定义的速度和加速度 0xA0: 回零运动, 利用内部 参数 0xA1: 回零运动, 利用下传 参数 0xA2: 取消回零并停止

			0x90: 重启暂停的轴 0x82: 暂停轴 0x91: 退出轴运动 AxisD: 轴号 0、1、2、3 SpdDir: 速度方向 Dummy: 占位 PosCmd: 指令位置 VelCmd: 指令速度 AccCmd: 指令加速度
4	0xF8 清轴位置值为 0	<pre>struct stMcCmd_ResetPosTyp{ uint8 SubCmd; uint8 AxisID; uint16 dummy; float PosCmd; }*pRestPosCmd;</pre>	AxisID: 轴号 0、1、2、3
5	0xFE 保存参数		
6	0xFC 清编码器值	<pre>struct stSetEncoderCmd{ uint8 AxisID; uint8 dummy1; uint16 dummy2; int32 EnCoderVal; }*pSetEnCoderCmd;</pre>	AxisD: 轴号 0、1、2、3 EnCoderVal: 编码器新值
7	0xF3 下传 G 指令控制	<pre>struct stGCodeCmmndType{ uint16 IntrpltType;</pre>	IntrpltType: 0x81 增量式直线联动插补、0x82/0x83 增量式

		<pre> uint16 AxisEn; float TargtPosX; float TargtPosY; float TargtPosZ; float TargtPosT; float CmmndVelX; float CmmndVelY; float CmmndVelZ; float CmmndVelT; float TargtAccX; float TargtAccY; float TargtAccZ; float TargtAccT; float Radius; float DelayCmmnd; }; </pre>	圆弧插补、增量式各轴单插补、0x01 绝对式直线联动插补、0x02/0x03 绝对式圆弧插补 AxisEn: 轴命使能标识 0bit=1 X 使能; 1bit=1 Y 使能; 2Bit=1 Z 使能; 3bit=1 T 使能 TargtPosX: 1 轴位置 下同 CmmndVelX: 1 轴速度 下同 TargtAccX: 1 轴加速度 下同 Radius: 圆半径 DelayCmmnd: 延时时间
8	0xF6 查询	<pre> struct stAskMoveInfoCmndTyp{ uint8 AxisId; uint8 ParaIdx; }*pAskMovInfoCmnd; </pre>	AxisD: 轴号 0、1、2、3 ParaIdx: 欲查询内容序号

2 上传数据格式

2.1 数据结构

控制卡在接收到指令后，会回传指令执行结果和数据（如果有）至上位机，上传格式如下所示。

```
struct stUploadNcSttType{
    uint8 u8CmndtType;      ///< 下发的命令
    uint8 u8SubCmnd;        ///< 下发的子指令
    uint8 u8Len;            ///< 要上传的数据长度
    uint8 u8CmndStt;        ///< 下发的命令的执行结果
    uint8 u8Buf[CMNDBUFSIZE]; ///< 数据区，
};
```

指令执行结果如下所示

```
enum eMcuResponseSttTyp{
    McuRespnsStt_NoErr      = 0x00,      ///< 无错误
    McuRespnsStt_SysRunning = 0x01,      ///< 系统正在运行中
    McuRespnsStt_IdxErr     = 0x02,      ///< 参数索引值有误
    McuRespnsStt_CmndTypeErr= 0x03,      ///< 下发的指令有误，不支持
    McuRespnsStt_OutptIdx   = 0x04,      ///< 下发的输出序号有误
    McuRespnsStt_OpIdx      = 0x05,      ///< 手动操作的子指令有误
    McuRespnsStt_SumErr     = 0x06,      ///< 校验合有误
    McuRespnsStt_ResetSumCmd= 0x07,      ///< 重置位置的子指令有误
};
```

表格 2 上传数据格式

在下发指令没有要求数据时，上传的结构中不包括数据区，但要多加一个校验和。

序号	命令	子命令	u8Buf 格式
1	0xF6	0x00 查询当前位置	fPara
		0x01 查询当前速度	fPara
		0x03 查询当前编码器值	Int32
		0x05 查询输入	UINT32
		0x0B 查询运动状态	Int32 .0 位表示是否在运动，0 停止，1 运动；.1 位取反表示 g 指令执行完